1. Problem Title & Source

Title: **Combine Candies – Minimum Time**

Source: **TCS NQT Previous Coding Questions**

2. Problem Statement (Clear Version)

A candy factory produces candies in **different packets**, where each packet contains a certain number of candies.

To ship them efficiently, the factory combines packets together.

Rule for combining packets:

- Two packets can be combined into one larger packet.
- The **time required to combine two packets** is equal to the **sum of candies in both packets**.

The factory wants to **combine all packets into a single packet**.

Your task is to determine the **minimum total time required to combine all packets**.

Important idea:

To minimize the total cost, we should **always combine the two smallest packets first**.

3. Examples

Example 1

Input

4

4 3 2 6

Output

29

Explanation

Packets

4, 3, 2, 6

Step 1

2 + 3 = 5

time = 5

Packets now

4, 5, 6

Step 2

4 + 5 = 9

time = 9

Packets now

6, 9

Step 3

6 + 9 = 15

time = 15

Total time

5 + 9 + 15 = 29

Example 2

Input

3

1 2 3

Output

9

Explanation

Step 1

1 + 2 = 3

Step 2

3 + 3 = 6

Total

3 + 6 = 9

4. Constraints

1 ≤ N ≤ 1000

1 ≤ arr[i] ≤ 10^6

Observations

- Combining larger packets early increases cost
- Optimal strategy → combine **smallest packets first**

5. Core Concept (Pattern)

This problem uses:

Greedy strategy

Min Heap (Priority Queue)

It is identical to the famous problem:

Minimum Cost to Connect Ropes

6. Thought Process

Brute Force Idea

Try all possible combinations of packets.

But the number of combinations grows very fast.

Time complexity becomes exponential.

Greedy Insight

Always combine the **two smallest packets first**.

Why?

Because combining larger packets early leads to **higher future costs**.

This greedy choice minimizes the total cost.

7. Visualization

Example

Packets = [4,3,2,6]

Min Heap

2,3,4,6

Steps

Combine	Cost	Remaining
2 + 3	5	4,5,6
4 + 5	9	6,9
6 + 9	15	15

Total

29

8. Pseudocode

```
read N
read packets

insert packets into minHeap

totalCost = 0

while heap size > 1

   a = extract min
   b = extract min

   cost = a + b
```

```
    totalCost += cost

    insert cost back into heap

print totalCost
```

9. Code Implementation

Python

```
import heapq

n = int(input())
arr = list(map(int,input().split()))

heapq.heapify(arr)

total = 0

while len(arr) > 1:

    a = heapq.heappop(arr)
    b = heapq.heappop(arr)

    cost = a + b

    total += cost

    heapq.heappush(arr, cost)

print(total)
```

Java

```
import java.util.*;

class Main {

    public static void main(String[] args){
```

```
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        PriorityQueue<Integer> pq = new PriorityQueue<>();

        for(int i=0;i<n;i++)
            pq.add(sc.nextInt());

        int total = 0;

        while(pq.size() > 1){

            int a = pq.poll();
            int b = pq.poll();

            int cost = a + b;

            total += cost;

            pq.add(cost);
        }

        System.out.println(total);
    }
}
```

10. Time & Space Complexity

Time Complexity

Heap operations

O(log N)

Total operations

O(N log N)

Space Complexity

O(N)

Heap stores packet sizes.

11. Common Mistakes

1. Combining packets randomly
2. Sorting once instead of using heap
3. Forgetting to reinsert combined packet
4. Using max heap instead of min heap

12. Detailed Dry Run

Input

4

4 3 2 6

Heap

2 3 4 6

Step 1

2 + 3 = 5

total = 5

Heap

4 5 6

Step 2

4 + 5 = 9

total = 14

Heap

6 9

Step 3

6 + 9 = 15

total = 29

Output

29

13. Real-Life Applications

This pattern appears in:

- Data compression (Huffman coding)
- File merging
- Rope connection problems
- Task scheduling optimization

14. Common Traps

1. Sorting array repeatedly
2. Missing heap reinsertion
3. Using wrong heap type
4. Not stopping when heap size = 1

15. Related Problems

Heap / Greedy problems:

LC 1167 Minimum Cost to Connect Sticks

LC 215 Kth Largest Element

LC 347 Top K Frequent Elements

16. Alternate Approaches

Approach	Complexity	Notes
Brute force	Exponential	infeasible
Heap greedy	O(N log N)	optimal

17. Why This Solution Works

Combining the **two smallest packets first** ensures that small costs are added earlier and larger costs later. This greedy choice minimizes the cumulative cost of all combinations.

18. Variations / Follow-Up Questions

Possible interview variations:

1. Combine ropes with minimum cost
2. Minimum cost to merge files
3. Huffman coding tree construction
4. Connect sticks with minimum cost